

Food Store

Diseño e Implementación de una Base de Datos Relacional
para la Gestión de Pedidos de Comida

Trabajo Práctico Integrador
Base de Datos

Tecnicatura Universitaria en Programación

Docente:

Carlos Yácomo

Integrantes:

Daniela Díaz

Lautaro Fernández

Danilo Serrano

Jesus Ramírez

Elio Marí

Agosto 2026

Índice

1. Introducción
2. Modelo Conceptual (DER)
3. Modelo Lógico y Normalización
4. Modelo Físico (DDL)
5. Lógica de Servidor
6. Historias de Usuario
7. Consultas Analíticas
8. Transacciones y Concurrencia
9. Decisiones de Diseño
10. Instrucciones de Ejecución
11. Conclusiones
12. Referencias

1. Introducción

En este trabajo implementamos la base de datos relacional para Food Store, un sistema de gestión de pedidos de comida. El objetivo fue diseñar el modelo de datos completo (conceptual, lógico y físico), implementarlo en PostgreSQL y demostrar el uso de SQL avanzado: vistas, funciones, triggers, procedimientos almacenados y control de transacciones.

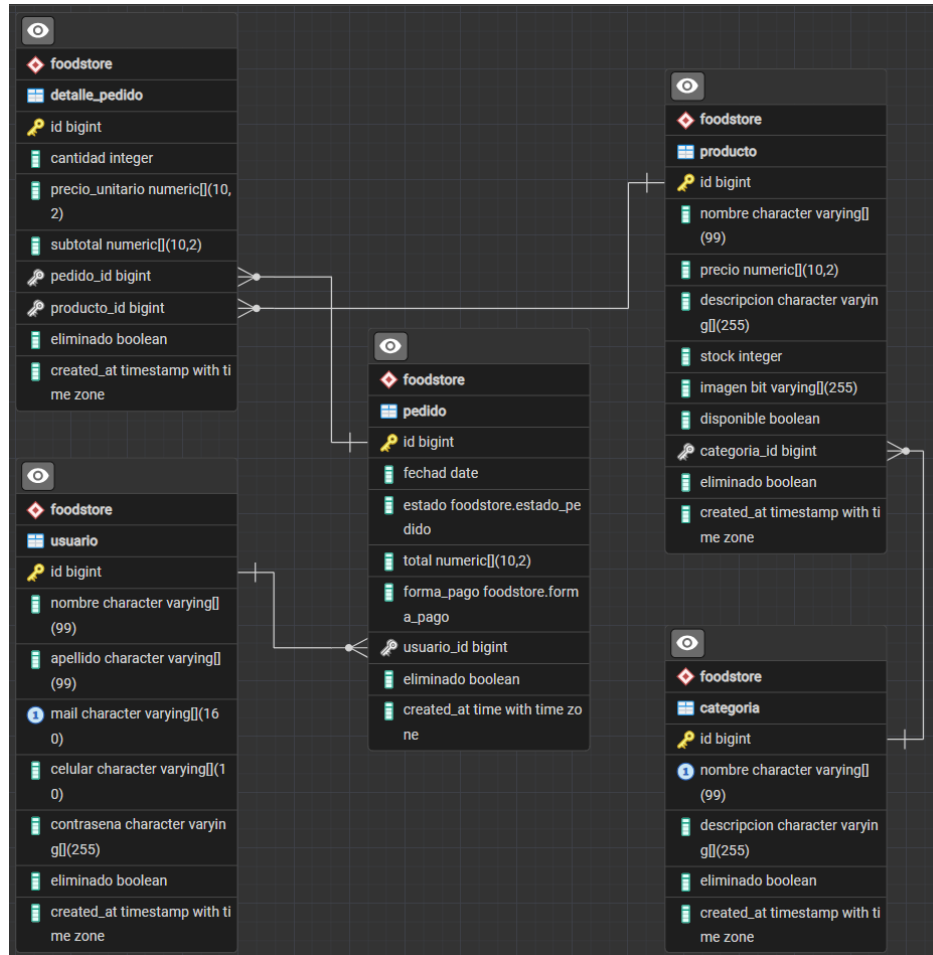
El sistema maneja cinco entidades principales (categorías, productos, usuarios, pedidos y detalles de pedido) y soporta operaciones ABM con borrado lógico, cálculo automático de totales mediante triggers y un procedimiento transaccional para la creación de pedidos con control de stock y concurrencia.

Toda la interacción con el sistema se realiza mediante sentencias SQL ejecutadas directamente sobre el motor PostgreSQL. No se desarrolló interfaz de usuario ni aplicación externa.

Aspecto	Detalle
Motor de BD	PostgreSQL 16+
Lenguaje	SQL estándar / PL/pgSQL
Herramientas	psql / pgAdmin / pgModeler
Entregables	Scripts SQL, DER, documentación

2. Modelo Conceptual (DER)

El siguiente diagrama muestra el modelo entidad-relación en notación de pata de gallo, generado con el modelador de PostgreSQL:



Entidades y atributos

Entidad	Atributos principales	Relaciones
categoria	id, nombre, descripcion	1:N con producto
producto	id, nombre, precio, stock, imagen, disponible, categoria_id	N:1 con categoria 1:N con detalle_pedido
usuario	id, nombre, apellido, mail, celular, contrasena, rol	1:N con pedido
pedido	id, fecha, estado, total, forma_pago, usuario_id	N:1 con usuario 1:N con detalle_pedido

detalle_pedido	id, cantidad, precio_unitario, subtotal, pedido_id, producto_id	N:1 con pedido N:1 con producto
----------------	---	------------------------------------

Enumerados del dominio

Tipo	Valores	Implementación
rol	ADMIN, USUARIO	CREATE TYPE rol AS ENUM (...)
estado_pedido	PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO	CREATE TYPE estado_pedido AS ENUM (...)
forma_pago	TARJETA, TRANSFERENCIA, EFECTIVO	CREATE TYPE forma_pago AS ENUM (...)

Preguntas guía

1. ¿Por qué la relación entre producto y pedido no puede resolverse como una relación binaria simple 1:N? ¿Qué información se perdería si lo intentaras?

Si pusiéramos una FK directa de producto a pedido (o viceversa), solo podríamos asociar un producto con un pedido, o un pedido con un solo producto. La realidad es que un pedido puede tener varios productos y un mismo producto puede aparecer en muchos pedidos: es una relación M:N.

Además, al momento de la compra necesitamos guardar información que NO pertenece ni al producto ni al pedido por separado: la cantidad pedida, el precio unitario en ese momento y el subtotal. Esta información es propia de la relación, no de las entidades. Si elimináramos la tabla intermedia detalle_pedido, perderíamos:

- La cantidad de cada producto en cada pedido.
- El precio congelado al momento de la compra (si el producto sube de precio después, el pedido viejo mantiene el precio original).
- El subtotal por línea (cantidad × precio_unitario).
- La posibilidad de que un pedido tenga múltiples productos distintos.

Por eso modelamos la relación como una entidad asociativa (detalle_pedido) con sus propios atributos, que resuelve la M:N y almacena los datos del contexto de la compra.

2. ¿Qué entidad tiene participación parcial en la relación con categoría, y cuál tiene participación total? ¿Qué pasaría si invirtieras esa lectura?

Producto tiene participación total en la relación con categoría: todo producto DEBE pertenecer a una categoría (categoria_id es NOT NULL). No puede existir un producto sin categoría asignada.

Categoría tiene participación parcial: puede existir una categoría que todavía no tenga productos asociados (por ejemplo, si la creamos para usarla después).

Si invirtiéramos la lectura (categoría total, producto parcial), significaría que toda categoría debe tener al menos un producto (no podríamos crear categorías vacías), y que un producto podría no tener categoría. Esto sería problemático porque:

- No podríamos preparar categorías de antemano sin tener productos cargados.
- Los productos sin categoría romperían el filtrado por categoría y las vistas que hacen JOIN con categoría.
- En nuestro schema, el NOT NULL en categoria_id refuerza exactamente la participación total de producto.

3. ¿Alguno de tus atributos podría descomponerse en partes más simples? Justificá tu decisión de dejarlo compuesto o de separarlo.

El caso más claro es el nombre del usuario. En nuestro modelo YA está separado en nombre y apellido (dos columnas independientes en la tabla usuario). Esto nos permite:

- Ordenar por apellido en listados.
- Buscar por nombre o apellido por separado.
- Concatenar nombre || ' ' || apellido cuando necesitamos el nombre completo (lo hacemos en la vista v_pedidos_resumen).

Si lo hubiéramos dejado como un solo campo "nombre_completo", sería mucho más difícil hacer estas operaciones porque tendríamos que usar funciones de texto para separar las partes.

Otro atributo que podría discutirse es la dirección del usuario, pero en nuestro modelo no la incluimos porque el enunciado no la requiere. Si la agregáramos, deberíamos evaluar si separarla en calle, número, ciudad, etc.

Los demás atributos (precio, stock, fecha, etc.) son valores atómicos que no tienen sentido descomponer. El mail y el celular son cadenas que, si bien tienen estructura interna (dominio del mail, código de área del teléfono), en la práctica se manejan como valores completos y no necesitamos operar sobre sus partes.

3. Modelo Lógico y Normalización

Del DER derivamos el esquema relacional. Cada relación se expresa en notación formal, subrayando la clave primaria:

```
categoria(id, nombre, descripcion, eliminado, created_at)

producto(id, nombre, precio, descripcion, stock, imagen, disponible,
         categoria_id -> categoria, eliminado, created_at)

usuario(id, nombre, apellido, mail, celular, contrasena, rol,
        eliminado, created_at)

pedido(id, fecha, estado, total, forma_pago,
       usuario_id -> usuario, eliminado, created_at)

detalle_pedido(id, cantidad, precio_unitario, subtotal,
              pedido_id -> pedido, producto_id -> producto, eliminado, created_at)
```

Justificación de formas normales

Forma Normal	Justificación
1FN	Todos los atributos son atómicos. Los productos de un pedido no se guardan como lista en pedido, sino en la tabla detalle_pedido.
2FN	Con clave primaria surrogada (id), todo atributo no clave depende de la clave completa. Si usáramos la clave compuesta (pedido_id, producto_id) en detalle_pedido, igualmente los demás atributos dependen de ambas columnas.
3FN	No hay dependencias transitivas. Por ejemplo, el nombre de la categoría no se almacena en producto, solo categoria_id (la FK).
BCNF	Todo determinante es superclave. En usuario, mail es clave candidata (UNIQUE), pero la PK id es superclave. No hay dependencias donde el determinante no sea superclave.

Decisión sobre el atributo total

El atributo total del pedido es un dato derivado (total = suma de subtotales). Decidimos almacenarlo en la tabla pedido y mantenerlo actualizado automáticamente mediante un trigger AFTER en detalle_pedido. Esto es una desnormalización controlada que se justifica por rendimiento: evita recalcular el total en cada consulta de listado de pedidos.

4. Modelo Físico (DDL)

El esquema físico se implementa en el archivo schema.sql. A continuación se muestra un fragmento con las decisiones principales:

```
CREATE TYPE rol AS ENUM ('ADMIN', 'USUARIO');
CREATE TYPE estado_pedido AS ENUM ('PENDIENTE', 'CONFIRMADO',
    'TERMINADO', 'CANCELADO');
CREATE TYPE forma_pago AS ENUM ('TARJETA', 'TRANSFERENCIA', 'EFECTIVO');

CREATE TABLE producto (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(99) NOT NULL,
    precio NUMERIC(10,2) NOT NULL CHECK (precio >= 0),
    descripcion VARCHAR(255),
    stock INTEGER NOT NULL DEFAULT 0 CHECK (stock >= 0),
    imagen VARCHAR(255),
    disponible BOOLEAN NOT NULL DEFAULT TRUE,
    categoria_id BIGINT NOT NULL REFERENCES categoria(id),
    eliminado BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Restricciones implementadas

Regla de negocio	Mecanismo
Precio y stock no negativos	CHECK (precio >= 0), CHECK (stock >= 0)
Cantidad de detalle > 0	CHECK (cantidad > 0)
Mail de usuario único	UNIQUE (mail)
No repetir producto en un pedido	UNIQUE (pedido_id, producto_id)
Pedido siempre con usuario	usuario_id NOT NULL + FK
No borrar pedido con detalles	ON DELETE RESTRICT

Índices

Índice	Tabla	Justificación
idx_producto_categoria_id	producto(categoria_id)	Listado de productos por categoría
idx_pedido_usuario_id	pedido(usuario_id)	Historial de pedidos por usuario

idx_producto_no_eliminado	producto(nombre) WHERE eliminado=FALSE	Búsqueda de productos vigente
---------------------------	--	-------------------------------

Una pequeña demostración de la demora que supone la presencia de índices. Si seleccionamos todos los productos no eliminados, el tiempo de ejecución del SELECT es el siguiente:

Data Output		Messages	Notifications
≡ + SQL Showing rows: 1 to 6			
	QUERY PLAN text		
1	Seq Scan on producto (cost=0.00..1.12 rows=6 width=1294) (actual time=0.012..0.013 rows=11.00 loop...		
2	Filter: (NOT eliminado)		
3	Rows Removed by Filter: 1		
4	Buffers: shared hit=1		
5	Planning Time: 0.048 ms		
6	Execution Time: 0.022 ms		
Total rows: 6		Query complete 00:00:00.049	

Si, por otro lado, ejecutamos habiendo creado un índice para el caso específico, tenemos que de hecho tarda más. Esto se debe a que la cantidad de datos en la tabla productos es pequeña (tan solo once), por lo que el tiempo que tarda PostgreSQL en buscar el índice adecuado para el SELECT no compensa el tiempo que se tarda en ejecutar el comando en su completitud.

Data Output

Messages

Notifications

≡ +

▼

▼

SQL

Showing rows: 1 to 8

	QUERY PLAN text
3	Rows Removed by Filter: 1
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=16 read=1
7	Planning Time: 1.944 ms
8	Execution Time: 0.041 ms

5. Lógica de Servidor

Vistas

Vista	Descripción
v_categorias_vigentes	Categorías con eliminado = FALSE
v_productos_vigentes	Productos vigentes con nombre de categoría (JOIN)
v_pedidos_resumen	Pedidos: id, usuario completo, fecha, estado, total
v_pedido_detalle	Líneas de un pedido: producto, cantidad, precio, subtotal

Ejemplo de vista implementada:

```
CREATE VIEW v_productos_vigentes AS
SELECT p.id, p.nombre, p.precio, p.stock,
       c.nombre AS categoria
FROM   producto p
JOIN   categoria c ON c.id = p.categoria_id
WHERE  p.eliminado = FALSE AND c.eliminado = FALSE;
```

Función de cálculo de total

Análoga al método `calcularTotal()` de la interfaz `Calculable de Programación 2`. Recibe un pedido y devuelve la suma de sus subtotales:

```
CREATE OR REPLACE FUNCTION calcular_total_pedido(p_pedido_id BIGINT)
RETURNS NUMERIC(10,2) AS $$
    SELECT COALESCE(SUM(subtotal), 0)
    FROM   detalle_pedido
    WHERE  pedido_id = p_pedido_id AND eliminado = FALSE;
$$ LANGUAGE sql STABLE;
```

Triggers

Implementamos tres triggers que automatizan el cálculo de importes:

Trigger	Evento	Función
trg_subtotal	BEFORE INSERT/UPDATE en detalle_pedido	Congela precio_unitario y calcula subtotal = cantidad x precio
trg_total_ins	AFTER INSERT en detalle_pedido	Recalcula el total del pedido (usa transition table)
trg_total_upd	AFTER UPDATE en detalle_pedido	Recalcula el total del pedido

El trigger BEFORE congela el precio del producto en el momento de la compra:

```
CREATE OR REPLACE FUNCTION fn_set_subtotal()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.precio_unitario IS NULL THEN
        SELECT precio INTO NEW.precio_unitario
        FROM producto WHERE id = NEW.producto_id;
    END IF;
    NEW.subtotal := NEW.cantidad * NEW.precio_unitario;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Procedimiento transaccional: sp_crear_pedido

Encapsula la creación de un pedido completo con validaciones y descuento de stock. Recibe el usuario, la forma de pago y un JSON con los productos y cantidades. Utiliza SELECT ... FOR UPDATE para evitar sobreventa concurrente.

Validaciones que realiza:

- Usuario existe y no está eliminado
- Cada producto existe, está disponible y tiene stock suficiente
- Si alguna validación falla, toda la transacción se revierte (rollback)

```
-- Ejemplo de uso:
CALL sp_crear_pedido(
    1,
    'EFFECTIVO'::forma_pago,
    '[{"producto_id":1,"cantidad":2},
    {"producto_id":7,"cantidad":1}]'::jsonb
);
```

6. Historias de Usuario

Implementamos 16 historias de usuario distribuidas en 4 épicas. Todas las consultas están en el archivo queries.sql y son ejecutables directamente contra la base de datos.

Épica 1: Gestión de Categorías

Código	Historia	Artefacto SQL
HU-CAT-01	Listar categorías	SELECT sobre v_categorias_vigentes
HU-CAT-02	Crear categoría	INSERT ... RETURNING id
HU-CAT-03	Editar categoría	UPDATE por id con filtro de eliminado
HU-CAT-04	Eliminar categoría	UPDATE eliminado = TRUE (baja lógica)

Épica 2: Gestión de Productos

Código	Historia	Artefacto SQL
HU-PROD-01	Listar productos	SELECT con JOIN a categoría + filtro
HU-PROD-02	Crear producto	INSERT validando categoría vigente
HU-PROD-03	Editar producto	UPDATE parcial con COALESCE
HU-PROD-04	Eliminar producto	UPDATE eliminado = TRUE

Épica 3: Gestión de Usuarios

Código	Historia	Artefacto SQL
HU-USR-01	Listar usuarios	SELECT con filtro de eliminado
HU-USR-02	Crear usuario	INSERT ... RETURNING id (UNIQUE mail)
HU-USR-03	Editar usuario	UPDATE por id
HU-USR-04	Eliminar usuario	UPDATE eliminado = TRUE

Épica 4: Gestión de Pedidos

Código	Historia	Artefacto SQL
HU-PED-01	Listar pedidos	SELECT sobre v_pedidos_resumen
HU-PED-02	Crear pedido	CALL sp_crear_pedido (transaccional)
HU-PED-03	Actualizar estado	UPDATE estado/forma_pago
HU-PED-04	Eliminar pedido	BEGIN + UPDATE eliminado en cascada

Ejemplo de HU-PED-02 (crear pedido con el procedimiento):

```
CALL sp_crear_pedido(  
    1, -- usuario_id (Miguel)  
    'EFFECTIVO'::forma_pago,  
    '[{"producto_id":1,"cantidad":2},  
     {"producto_id":7,"cantidad":1}]'::jsonb  
);  
  
-- Verificar:  
SELECT * FROM v_pedidos_resumen ORDER BY id DESC LIMIT 1;  
SELECT * FROM v_pedido_detalle  
WHERE pedido_id = (SELECT MAX(id) FROM pedido);
```

7. Consultas Analíticas

Implementamos cinco consultas analíticas que demuestran SQL avanzado. Todas están en `queries.sql`.

A) Top 5 productos más vendidos

```
SELECT pr.id, pr.nombre, SUM(dp.cantidad) AS unidades
FROM   detalle_pedido dp
JOIN   producto pr ON pr.id = dp.producto_id
WHERE  dp.eliminado = FALSE
GROUP BY pr.id, pr.nombre
ORDER BY unidades DESC
LIMIT 5;
```

B) Facturación por categoría y mes

```
SELECT c.nombre AS categoria,
       date_trunc('month', ped.fecha)::DATE AS mes,
       SUM(dp.subtotal) AS facturado
FROM   detalle_pedido dp
JOIN   pedido ped ON ped.id = dp.pedido_id
JOIN   producto pr  ON pr.id = dp.producto_id
JOIN   categoria c  ON c.id = pr.categoria_id
WHERE  dp.eliminado = FALSE AND ped.eliminado = FALSE
GROUP BY c.nombre, date_trunc('month', ped.fecha)
ORDER BY mes DESC, facturado DESC;
```

C) Ranking de usuarios por gasto (función de ventana)

```
SELECT u.id, u.nombre || ' ' || u.apellido AS usuario,
       SUM(ped.total) AS gasto,
       RANK() OVER (ORDER BY SUM(ped.total) DESC) AS puesto
FROM   pedido ped
JOIN   usuario u ON u.id = ped.usuario_id
WHERE  ped.eliminado = FALSE
GROUP BY u.id, u.nombre, u.apellido
ORDER BY puesto;
```

D) Pedidos sobre el promedio (subconsulta)

```
SELECT id, total
FROM   pedido
WHERE  eliminado = FALSE
       AND total > (SELECT AVG(total) FROM pedido WHERE eliminado = FALSE)
ORDER BY total DESC;
```

E) Productos sin ventas (LEFT JOIN + IS NULL)

```
SELECT pr.id, pr.nombre
FROM   producto pr
LEFT   JOIN detalle_pedido dp ON dp.producto_id = pr.id AND dp.eliminado = FALSE
WHERE  pr.eliminado = FALSE AND dp.id IS NULL
ORDER BY pr.id;
```

8. Transacciones y Concurrency

Demostramos los conceptos de control transaccional con escenarios ejecutables en transacciones.sql.

Atomicidad (rollback automático)

Al llamar a `sp_crear_pedido` con un producto inexistente, el procedimiento lanza una excepción y toda la transacción se revierte. No queda ni el pedido ni ningún detalle.

```
-- Esto falla: producto 999 no existe
CALL sp_crear_pedido(1, 'EFECTIVO'::forma_pago,
  '[{"producto_id":999,"cantidad":2}]'::jsonb);
-- ERROR: Producto 999 inexistente o eliminado
-- El pedido NO se creó (rollback automático)
```

Al llamar a `sp_crear_pedido` con un producto inexistente, el procedimiento lanza una excepción y toda la transacción se revierte. No queda ni el pedido ni ningún detalle.

Transacción manual (BEGIN / COMMIT / ROLLBACK)

Mostramos cómo una secuencia de operaciones se confirma con `COMMIT` o se revierte con `ROLLBACK`:

```
BEGIN;

UPDATE detalle_pedido SET eliminado = TRUE WHERE pedido_id = 2;

UPDATE pedido SET eliminado = TRUE WHERE id = 2;

-- Con este commit, los cambios son aplicados permanentemente.

COMMIT;

SELECT * FROM pedido
WHERE eliminado = TRUE;
```

Data Output Messages Notifications									
Showing rows: 1 to 3 Page No: 1									
	id [PK] bigint	cantidad integer	precio_unitario numeric (10,2)	subtotal numeric (10,2)	pedido_id bigint	producto_id bigint	eliminado boolean	created_at timestamp with time zone	
1	3	3	150.00	450.00	2	4	true	2026-08-27 15:01:33.773331+...	
2	4	2	140.00	280.00	2	5	true	2026-08-27 15:01:33.773331+...	
3	5	1	250.00	250.00	2	8	true	2026-08-27 15:01:33.773331+...	

Data Output Messages Notifications									
Showing rows: 1 to 1 Page No: 1									
	id [PK] bigint	fecha date	estado estado_pedido	total numeric (10,2)	forma_pago forma_pago	usuario_id bigint	eliminado boolean	created_at timestamp with time zone	
1	2	2026-08-...	PENDIENTE	0.00	TARJETA	3	true	2026-08-27 15:01:06.794107+...	

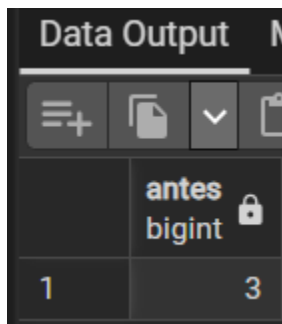
Un ejemplo en que se emplea ROLLBACK

```
SELECT COUNT(*) AS antes FROM pedido;

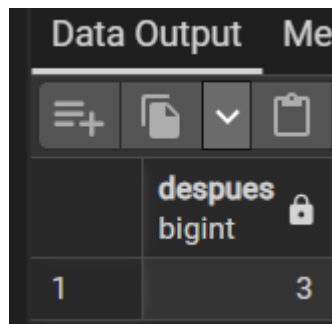
BEGIN;
    INSERT INTO pedido (usuario_id, forma_pago)
    VALUES (5, 'TRANSFERENCIA'::forma_pago);

    -- Cambiamos de idea; revertimos todo.
ROLLBACK;

-- La cantidad no cambió
SELECT COUNT(*) AS despues FROM pedido;
```



Data Output	
1	3



Data Output	
1	3

Concurrencia y aislamiento

Para evitar sobreventa, `sp_crear_pedido` bloquea la fila del producto con `SELECT ... FOR UPDATE`. Si dos sesiones intentan comprar el mismo producto simultáneamente, la segunda espera a que la primera termine:

```
-- ESCENARIO 9
-- Para probar esto se necesitan DOS sesiones psql abiertas simultáneamente.
-- SESIÓN 1 (ejecutar primero):
BEGIN;
SELECT stock FROM producto WHERE id = 1 FOR UPDATE;
    -- (la fila queda bloqueada, esta sesión "no termina")
    -- Esperar unos segundos para que Sesión 2 intente...
    UPDATE producto SET stock = stock - 1 WHERE id = 1;
COMMIT;

-- SESIÓN 2 (ejecutar mientras Sesión 1 está bloqueando):
BEGIN;
SELECT stock FROM producto WHERE id = 1 FOR UPDATE;
-- ↑ Esta línea ESPERA hasta que Sesión 1 haga COMMIT
```

```

-- Recién ahí puede continuar y ve el stock ya actualizado.
UPDATE producto SET stock = stock - 1 WHERE id = 1;
COMMIT;

-- Resultado: ambas sesiones descuentan 1 unidad de forma segura,
-- sin sobreventa. El FOR UPDATE serializa el acceso a la fila.

-- ESCENARIO 10
-- PostgreSQL usa READ COMMITTED por defecto.
-- Con SERIALIZABLE, las transacciones se comportan como si se ejecutaran
-- una después de otra

-- Ejemplo teórico con dos sesiones:
-- SESIÓN 1 (READ COMMITTED):
    BEGIN;
    SELECT stock FROM producto WHERE id = 1; -- lee 15
        -- (Sesión 2 cambia el stock a 14 y hace COMMIT)
    SELECT stock FROM producto WHERE id = 1; -- lee 14 (lectura no repetible)
    COMMIT;

-- SESIÓN 1 (SERIALIZABLE):
    BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
    SELECT stock FROM producto WHERE id = 1; -- lee 15
    -- (Sesión 2 cambia el stock a 14 y hace COMMIT)
    SELECT stock FROM producto WHERE id = 1; -- SIGUE leyendo 15
    COMMIT;
    -- En SERIALIZABLE, la snapshot no cambia durante la transacción.

-- La diferencia es que READ COMMITTED ve los cambios de otras transacciones
-- después de cada sentencia, mientras que SERIALIZABLE ve una "foto"
-- del momento en que empezó la transacción.

```

Niveles de aislamiento

PostgreSQL usa READ COMMITTED por defecto. Con SERIALIZABLE, cada transacción ve una snapshot del momento en que empezó y no se ve afectada por cambios de otras transacciones.

Nivel	Comportamiento
READ COMMITTED	Cada sentencia ve los cambios commiteados hasta ese momento. Permite lecturas no repetibles.
SERIALIZABLE	La transacción ve una snapshot fija. Si otra transacción modifica los mismos datos, una de las dos falla.

9. Decisiones de Diseño

Soft delete (borrado lógico)

Todas las tablas tienen una columna eliminado BOOLEAN DEFAULT FALSE. Cuando se da de baja un registro, se marca eliminado = TRUE en lugar de borrarlo físicamente. Aunque se intente realizar un DELETE en una fila, un trigger previene que se eliminen los datos y automáticamente marca eliminado como TRUE. Las vistas filtran automáticamente WHERE eliminado = FALSE. Esto preserva el historial y la integridad referencial de los pedidos existentes.

Total materializado con trigger

Elegimos almacenar el total en pedido.total y mantenerlo actualizado con triggers AFTER. La alternativa era calcularlo siempre con la función calcular_total_pedido(), pero eso requiere un JOIN o subconsulta en cada listado de pedidos. El trigger sacrifica un poco de normalización a cambio de consultas más rápidas.

Precio congelado en detalle_pedido

El precio_unitario se copia del producto al momento de crear el detalle. Si después el producto cambia de precio, los pedidos anteriores mantienen el precio histórico. Esto es fundamental para la auditoría y la facturación correcta.

ON DELETE RESTRICT

La FK de detalle_pedido hacia pedido usa ON DELETE RESTRICT para impedir borrar un pedido que tenga detalles. Combinado con el soft delete, garantiza que los datos históricos siempre son consultables.

Atributos comunes (id, eliminado, created_at)

En Programación 2 todas las clases heredaban de una clase abstracta Base. En el modelo relacional no existe la herencia de implementación, así que estos atributos se repiten en cada tabla. Es la práctica habitual en bases de datos relacionales y evita la complejidad de INHERITS de PostgreSQL.

10. Instrucciones de Ejecución

Requisitos: PostgreSQL 14+ instalado con acceso a psql o pgAdmin.

Pasos para levantar la base de datos:

- 1. Crear la base de datos: createdb foodstore
- 2. Conectarse: psql -d foodstore
- 3. Ejecutar estructura: \i schema.sql
- 4. Ejecutar objetos: \i objects.sql
- 5. Cargar datos: \i data.sql
- 6. Probar consultas: \i queries.sql
- 7. Probar transacciones: \i transacciones.sql

Estructura de archivos:

Archivo	Contenido
schema.sql	Tipos ENUM, tablas, restricciones e índices
objects.sql	Vistas, funciones, triggers y procedimientos
data.sql	Datos de ejemplo (seed)
queries.sql	Consultas de historias de usuario + analíticas
transacciones.sql	Escenarios de transacciones y concurrencia
der.pgerd / der.png	Diagrama entidad-relación

11. Conclusiones

El proyecto nos permitió aplicar los conceptos de la materia en un sistema concreto. Partimos del modelo de Programación 2 y lo traducimos a un esquema relacional normalizado, implementando toda la lógica de negocio directamente en la base de datos.

Los puntos más importantes que trabajamos fueron:

- Diseñar un modelo ER y derivar el esquema relacional normalizado hasta BCNF.
- Implementar restricciones declarativas (CHECK, UNIQUE, FK) para garantizar la integridad.
- Usar triggers para automatizar cálculos (subtotal y total) que antes se hacían en Java.
- Encapsular la lógica de creación de pedidos en un procedimiento transaccional con control de concurrencia.
- Demostrar conceptos de transacciones, atomicidad y aislamiento con escenarios concretos.

Nuestra mayor dificultad fue encontrar la forma de testear el principio de aislamiento en una sola máquina, puesto que PgAdmin 4 no abre más de una ventana a la vez.

En general el proyecto quedó funcional, coherente y cumple con los requisitos del TPI.

12. Referencias

[PostgreSQL: Documentation: 16: Chapter 4. SQL Syntax.](#)

[PostgreSQL: Documentation: 16: Chapter 5. Data Definition.](#)

[PostgreSQL: Documentation: 16: Chapter 6. Data Manipulation.](#)

[PostgreSQL: Documentation: 16: 3.3. Foreign Keys.](#)

[PostgreSQL: Documentation: 16: Chapter 7. Queries.](#)

[PostgreSQL: Documentation: 16: Chapter 8. Data Types.](#)

[PostgreSQL: Documentation: 16: Chapter 9. Functions and Operators.](#)

[PostgreSQL: Documentation: 16: Chapter 10. Type Conversion.](#)

[PostgreSQL: Documentation: 16: Chapter 39. Triggers.](#)

[PostgreSQL: Documentation: 16: Chapter 40. Event Triggers.](#)

[PostgreSQL: Documentation: 18: 41.8. Transaction Management.](#)